

מס' שאלון - 530

8

ביולי 2020

מס' מועד 73

סמסטר 2020ב

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 4 שעות

בשאלון זה 12 עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

בהצלחה !!!

שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") בביטויים חדשים המאפשרים עבודה עם מערך. את המערך ניתן ליצור, ולאחר מכן לקרוא את ערכיו, אך לא לשנותם (להזכירכם, בכל השפות שהוגדרו בפרק 3 בספר, המשתנים היו immutable, ולכן המערך לא ניתן לשינוי לאחר הגדרתו)

להלן הדקדוק המגדיר את הביטויים החדשים לעבודה עם מערך:

Expression ::= array ({Expression}⁺)

array-exp (exps)

Expression ::= Expression[Expression]

index-exp (exp1 exp2)

ביטוי array-exp מורכב מרשימת ביטויים exps, שערכם מהווים את איברי המערך. הביטוי צריך להחזיר מערך עם איברים אלה. איברי המערך יכולים להיות מסוגים שונים.

ביטוי index-exp מורכב מ-2 ביטויים: exp1, exp2. כאשר ערכו של exp1 אמור להיות מערך וערכו של exp2 אמור להיות מספר המציין אינדקס של איבר במערך (האינדקסים ממוספרים החל מ-0). ביטוי index-exp צריך להחזיר את ערכו של האיבר הנמצא במערך באינדקס המבוקש. במקרה של אינדקס שגוי יש להדפיס הודעת שגיאה.

להלן דוגמא להמחשת השימוש בביטוי החדש:

```
let x= array (8, -(5,2), zero?(9))
in
-(x[0],x[1])
```

הסבר הדוגמה:

המשתנה x הוא מערך עם 3 איברים. כל התוכנית בדוגמה מחזירה את ההפרש בין האיבר הראשון במערך (8) לבין האיבר השני במערך (3) ולכן תוצאת התוכנית היא (num-val 5)

א. (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של של השפה כך שיתמוך בהגדרת הביטוי החדש.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% " (arbno (not #\newline)))) skip)
    (identifier (letter (arbno (or letter digit "_" "-" "?")))) symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")") diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression ("if" expression "then" expression "else" expression) if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression) let-exp)
  ))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת "ויהי" כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את הביטוי החדש.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

;;value-of : Exp x Env ----> ExpVal

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val (- num1 num2)))))
      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1) (bool-val #t) (bool-val #f)))))
      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
      (let-exp (var exp1 body)
        (let ((val1 (value-of exp1 env)))
          (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

בשפת PROC (שפת "וישגור") המוגדרת בספר, לפרוצדורות יש פרמטר יחיד. בשאלה זו, ברצוננו לשנות את אופן ההגדרה של פרוצדורה, כך שיהיה לה גם פרמטר שני עם ערך ברירת מחדל עבורו. כלומר, בזמן הפעלת פרוצדורה, ניתן לשלוח 1 או 2 פרמטרים לפרוצדורה. אם נשלח רק פרמטר אחד אזי הפרמטר השני של הפרוצדורה יקבל את ערך ברירת המחדל שהוגדר, ואם נשלחו 2 ארגומנטים, ערך ברירת המחדל לא יהיה בשימוש, והפרמטר השני של הפרוצדורה יקבל את הארגומנט השני שנשלח אליה.

להלן הדקדוק המעודכן עבור ביטוי הגדרת והפעלת פרוצדורה:

$Expression ::= \text{proc} (identifier, identifier = Expression) Expression$

`proc-exp (id1 id2 def body)`

$Expression ::= (Expression \{Expression\}^+)$

`call-exp (exp1 exps)`

כאשר, בביטוי proc-exp

- id1 הוא הפרמטר הראשון של הפרוצדורה
- id2 – הוא הפרמטר השני של הפרוצדורה
- def – ביטוי שערכו הוא ערך ברירת המחדל עבור הפרמטר השני
- body - הוא ביטוי המתאר את גוף הפרוצדורה שתיבנה.

ובביטוי call-exp,

- exp1 – הוא ביטוי שערכו הוא הפרוצדורה שרוצים להפעיל
- exps - רשימה של 1 או 2 ביטויים לכל היותר, שערכם משמשים כארגומנטים בהפעלת הפרוצדורה.

להלן דוגמה להמחשה:

```
let y = proc(a, b=50) -(a,b)
in
  -((y 70), (y 100 10))
```

הסבר: בדוגמה זו, y היא פרוצדורה עם 2 פרמטרים, כאשר לפרמטר השני יש ערך ברירת מחדל של 50. תוצאת כל התוכנית היא תוצאת ביטוי החיסור בין 2 תוצאות הפעלת הפרוצדורה. בהפעלה הראשונה, מסופק ארגומנט יחיד עם ערך 70, ולכן יוחזר מהפעלה זו 20 (50-70), תוצאת ההפעלה השנייה היא 90 (100-10), ולכן סה"כ תוצאת כל התוכנית בדוגמה היא: (num-val -70)

א. (7 נקודות)

להלן נתונים ה-scanner וה-parser של שפת "וישגור". כתבו מהו השינוי הדרוש בדקדוק של של השפה כך שיתמוך בהגדרת הביטוי החדש.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% (arbno (not #\newline))) skip)
    (identifier(letter (arbno (or letter digit "_" "-" "?")))
      symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)

    (expression (number) const-exp)
    (expression
      ("-" "(" expression "," expression ")")
    diff-exp)

    (expression
      ("zero?" "(" expression ")")
    zero?-exp)

    (expression
      ("if" expression "then" expression "else" expression)
    if-exp)

    (expression (identifier) var-exp)

    (expression
      ("let" identifier "=" expression "in" expression)
    let-exp)

    (expression
      ("proc" "(" identifier ")" expression)
    proc-exp)

    (expression
      "(" expression expression ")"
    call-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת "וישגור". ולאחריה פרוצדורת העזר *apply-procedure* כפי שמופיעה בספר הלימוד בעמוד 79. כתבו מה העדכון שנדרש להכניס על מנת לתמוך בחישוב של הביטוי החדש.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

;;value-of : Exp x Env→ExpVal

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val (- num1 num2))))))
      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1) (bool-val #t) (bool-val #f))))))
      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))
      (let-exp (var exp1 body)
        (let ((val1 (value-of exp1 env)))
          (value-of body (extend-env var val1 env))))

      (proc-exp (var body)
        (proc-val (procedure var body env)))

      (call-exp (rator rand)
        (let ((proc (expval->proc (value-of rator env)))
              (arg (value-of rand env)))
          (apply-procedure proc arg))))))

;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79

(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body (extend-env var val saved-env))))))
```

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם `foreach-exp`.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression ::= foreach identifier in ({identifier}^()) Expression*

<code>foreach-exp (id1 ids body)</code>

לביטוי `foreach-exp` המרכיבים הבאים:

- `id1` – שם של משתנה חדש עבור חישוב הביטוי.
- `ids` – רשימת שמות משתנים (שאמורים להיות מוגדרים כבר)
- `body` – ביטוי המשמש כגוף ה-`foreach`

ביטוי `foreach`, אמור לבצע פעולה זוהי על מספר משתנים, בכל פעם `id1` יתפקד כאחד המשתנים המוגדרים ע"י `ids` (החל מהראשון וכלה באחרון) ועבורו יש לבצע את `body`.

ביטוי `foreach` אינו מחזיר תשובה, לכן יש להחזיר ערך סתמי של `num-val 23` להלן דוגמא:

```
let a=3
in let b=4
  in let c= 5
    in
      begin
        foreach x in (a,b,c) x= -(x,1)
        -(a, -(0, -(b, -(0,c))))
      end
```

בדוגמה זו, מוגדרים 3 משתנים `a, b, c` עם ערכים 3, 4, 5 בהתאמה. ביצוע ביטוי `foreach` גורם להקטנת ערכו של כל אחד מהם ב-1, ל-2, 3, 4 בהתאמה. ערכה של כל התוכנית בדוגמה, הוא כערכו של הביטוי האחרון בבלוק ה-`begin` המחזיר את סכומם של המשתנים `a, b, c` ולכן התשובה המוחזרת היא (9 `num-val`)

א. (7 נקודות)

להלן נתונים ה-scanner וה-parser של שפת "וירמוז". כתבו מהו השינוי הדרוש בדקדוק של של השפה כך שיתמוך בהגדרת הביטוי החדש.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% (arbno (not #\newline))) skip)
    (identifier(letter (arbno (or letter digit "_" "-" "?")))
      symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")")
      diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression
      ("if" expression "then" expression "else" expression)
      if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression)
      let-exp)
    (expression ("proc" "(" identifier ")" expression)
      proc-exp)
    (expression "(" expression expression ")" call-exp)
    (expression
      ("letrec"
        (arbno identifier "(" identifier ")" "=" expression)
        "in" expression)
      letrec-exp)
    (expression ("set" identifier "=" expression)
      assign-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו-*apply-procedure* של שפת "וירמוז".
הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.
את התשובה כיתבו במחברת הבחינה.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))

      (var-exp (var) (deref (apply-env env var)))

      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val
             (- num1 num2))))))

      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1)
                (bool-val #t)
                (bool-val #f))))))

      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

      (let-exp (var exp1 body)
        (let ((v1 (value-of exp1 env)))
          (value-of body
                    (extend-env var (newref v1) env))))

      (proc-exp (var body)
        (proc-val (procedure var body env)))

      (call-exp (rator rand)
        (let ((proc (expval->proc (value-of rator env)))
              (arg (value-of rand env)))
```

```

        (apply-procedure proc arg)))

(letrec-exp (p-names b-vars p-bodies letrec-body)
  (value-of letrec-body
    (extend-env-rec* p-names b-vars p-bodies env)))

(assign-exp (var exp1)
  (begin
    (setref!
      (apply-env env var)
      (value-of exp1 env))
    (num-val 27)))))

(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body
          (extend-env var (newref val) saved-env))))))

```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p= proc (y:?)
  if let x= zero?(y)
  in
    if x then zero?(9) else zero? (0)
  then y else -(y,1)
in (p 7)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !